Davante

Desarrollo de Aplicaciones Web
Desarrollo Web Entornos Cliente

Tema 6.
Estructura de datos. El array

Diego Rodero Pulido

Tarea 6. Objetos y arrays

Tema 6. Estructura de datos. El array · Tarea individual · Calificación: 5 puntos por el trabajo + hasta 5 puntos en la defensa en clase.

Objetivo

Guardar los datos en objetos y en arrays, y trabajar con ellos: añadir, sacar, ordenar, filtrar, buscar y transformar. Al terminar la tarea debes ser capaz de:

- Crear objetos literales con propiedades y métodos, leerlas con punto y con corchetes, cambiarlas, borrarlas con `delete` y usar `this` dentro de un método.
- Añadir y sacar elementos de un array con `push`, `pop` y `shift`, y recorrerlo con `forEach`.
- Ordenar con `sort` y una función de comparación, filtrar con `filter`, buscar con `find`, `indexOf` e `includes`, y transformar con `map` y `reduce`.
- Distinguir los métodos que cambian el array original (`push`, `splice`, `sort`…) de los que devuelven uno nuevo (`slice`, `map`, `filter`…).

Qué vas a construir

En la carpeta `tema06` de tu repositorio de DWEC vas a completar una página, «Objetos y arrays», con seis ejercicios, cada uno en su tarjeta. **El HTML te lo doy hecho**; los datos y las funciones van en `js/app.js`, y los resultados se pintan en la página como en el tema 5.

- Está en la carpeta `tema06_plantilla` del GitHub de la asignatura: https://github.com/DRodero/DWEC_2627. Cópiala en tu repositorio y cámbiale el nombre a `tema06`. Trae esto:

```
tema06_plantilla/
├── index.html     la página, ya maquetada con Bootstrap
├── js/
│   └── app.js     los datos, una función por botón, modelos y TODO
├── capturas/      imágenes de relleno con el nombre de cada captura
└── README.md      el guion del README, con sus apartados
```

- Todo lo que va entre **[corchetes]** es un hueco: sustitúyelo por lo tuyo. Y cuando completes un `TODO`, borra el comentario.

Requisitos:

- Tu **nombre y apellidos** en la `navbar`, en el `<title>` y en el pie de la página.

- No cambies los `id` ni el `onclick` de los botones. Puedes mejorar el aspecto con Bootstrap, pero manteniéndolos.
- Los resultados se pintan con `createElement`, `textContent`, `classList` y `append`, como en el tema 5. Antes de volver a pintar una lista, vacíala con `textContent = ""`. Nada de `innerHTML`.
- **Sin funciones flecha:** todas las funciones, también las que pasas a `forEach`, `sort`, `map`, `filter`, `find` o `reduce`, se escriben con `function`. Las flechas, en el tema 7.
- `let` y `const`, nunca `var`; comparaciones con `===`; comillas rectas; y cada función deja su rastro en la consola con `console.log()`.

Ejercicio 1. Ficha de estudiante

1. Completa el objeto `estudiante` con tus datos (`nombre`, `apellidos`, `edad` y `curso`) y un método `presentarse()` que devuelva, usando `this`, una frase como «Soy Lucía Martín Ortega, tengo 20 años y estudio 2.º DAW.».
2. «Ver ficha» pinta en `#salida1` una lista con las cuatro propiedades y, debajo, la frase de `presentarse()`. Para la lista, guarda las claves en un array, recórrelo y lee cada valor con corchetes: `estudiante[clave]`.
3. «Cumplir años» suma 1 a la edad y «Quitar el curso» borra la propiedad con `delete`; las dos vuelven a pintar la ficha. Explica en el README qué sale en el curso después de borrarlo y por qué.

Ejercicio 2. Colores

1. El array `colores` empieza con tres colores. `mostrarColores()` y `anadirColor()` vienen hechas como modelo: léelas antes de seguir, porque el resto de la tarea se pinta igual.
2. «Quitar el último» saca el último color con `pop()` y «Quitar el primero», el primero con `shift()`. Las dos vuelven a pintar la lista y dicen en la consola qué color ha salido.
3. Si la lista está vacía, no quitan nada y muestran el aviso `#aviso2` con `avisar2()`. El contador `#totalColores` se actualiza siempre.

Ejercicio 3. Catálogo de productos

1. La plantilla trae el array `productos`, con seis objetos (`nombre`, `precio` y `categoria`), y la función `pintarProductos(lista)`, que pinta en la tabla la lista que le pases. «Todos» pinta el catálogo completo y dice en `#mensaje3` cuántos productos hay.
2. «Ordenar por precio» los ordena de más barato a más caro con `sort` y una función de comparación. Ordena una copia (`[...productos]`), no el original, y explica en el README por qué.
3. «Más de 50 €» filtra con `filter` y dice cuántos son. «Buscar» encuentra con `find` el primer producto cuyo nombre contiene lo escrito, sin distinguir mayúsculas (`toLowerCase` e `includes`). Si no hay ninguno, `find` devuelve `undefined`: dilo en `#mensaje3` y deja la tabla vacía.

Ejercicio 4. Notas

1. Completa el objeto `alumno` con dos métodos: `anadirNota(nota)`, que guarda la nota en `this.notas` y devuelve `true` si es un número entre 0 y 10, o devuelve `false` sin guardarla; y `media()`, que calcula la media con `reduce` y devuelve 0 si no hay notas.
2. «Añadir nota» lee `#nota`, la convierte con `Number()` y llama a `alumno.anadirNota()`. Si la casilla está vacía o la nota no es válida, muestra el error `#error4`. Ojo: `Number("")` vale 0, así que la casilla vacía se comprueba antes de convertir.
3. Las notas se pintan como badges, en verde (`text-bg-success`) si son de 5 o más y en rojo (`text-bg-danger`) si no, y debajo la media con dos decimales. «Borrar» deja la lista de notas vacía.

Ejercicio 5. map, filter y reduce

1. Con el array `numeros`, que va del 1 al 10, «Doble» crea con `map` un array con el doble de cada número, «Pares» se queda con los pares usando `filter` y «Suma» los suma con `reduce`.
2. Cada botón escribe en `#salida5` el resultado y, debajo, `numeros`, para que se vea que el original no ha cambiado.
3. «Los tres seguidos» encadena los tres pasos: los pares, sus dobles y la suma de esos dobles, con una línea por paso. El resultado tiene que ser 60.

Ejercicio 6. Ciudades: slice, splice y búsqueda

1. Las ocho ciudades se pintan como badges con su posición: «0 · Granada», «1 · Sevilla»… Pista: `forEach` le pasa a tu función el elemento y su posición.
2. «slice(2, 5)» enseña en `#salida6` lo que devuelve y cuántas ciudades siguen en el array. «splice(2, 2)» quita dos, enseña cuáles y vuelve a pintar; si quedan menos de tres, avisa de que hay que restaurar. «Restaurar» vuelve a copiar `ciudadesOriginal` en `ciudades`.
3. «Buscar» escribe lo que devuelven `includes()` e `indexOf()` con el nombre escrito. Busca la misma ciudad antes y después del `splice`, y fíjate en su posición.

Evidencias y reflexión (README.md)

En el `README.md` de la carpeta explicas lo que has hecho y lo demuestras; la plantilla trae el guion con los apartados. Las capturas van en la carpeta `capturas/`, tienen que estar hechas en tu equipo y cada una lleva debajo una o dos líneas que expliquen qué se ve.

1. Seis capturas propias, que sustituyen a las imágenes de relleno de `capturas/` con el mismo nombre: la página recién cargada, con tu nombre visible; la ficha y los colores después de usarlos; el catálogo ordenado por precio; cinco notas o más con su media; los tres pasos del ejercicio 5; y las ciudades después de `splice` y de una búsqueda.
2. Una reflexión de 5 a 8 líneas: ¿qué métodos de los que has usado cambian el array original y cuáles devuelven uno nuevo? Pon como ejemplo lo que pasa en el ejercicio 6 con `slice` y con `splice`, y explica por qué en el ejercicio 3 se ordena una copia.
3. Fuentes consultadas, con su enlace, y el apartado «Uso de IA» si corresponde.

Entrega

- Todo va en la carpeta `tema06` de tu repositorio público de DWEC, con esta estructura:

```
tema06/
├── index.html     la página de la plantilla, con tu nombre
├── js/
│   └── app.js     los datos y una función por botón
├── capturas/      las seis capturas
└── README.md      capturas, reflexión y fuentes
```

- Sube el trabajo en varios commits a medida que avances (tres como mínimo), con mensajes que digan qué cambia: «Ejercicio 1: ficha de estudiante», «Catálogo ordenado por precio»…
- En la tarea del campus virtual entregas solo la URL de la carpeta: `https://github.com/tu-usuario/tu-repositorio/tree/main/tema06`

- Extensión orientativa del README: de 200 a 400 palabras, más las capturas.
- Se corrige el último commit anterior a la fecha límite publicada en la propia tarea del campus; lo que subas después no cuenta. No se admiten entregas por otros medios.

Calificación

El trabajo vale 5 puntos, repartidos por ejercicio. Los otros 5 puntos, hasta el 10, se obtienen en la defensa en clase.

Apartado	Puntos	Qué se valora
Ejercicio 1. Ficha de estudiante	0,5	El objeto con sus cuatro propiedades y presentarse() con this; la ficha se pinta leyendo con corchetes y cambia al cumplir años y al borrar el curso.
Ejercicio 2. Colores	0,5	pop y shift vuelven a pintar la lista y mantienen el contador; aviso cuando la lista está vacía.
Ejercicio 3. Catálogo de productos	0,75	sort con función de comparación sobre una copia, filter con el recuento y find con el caso de no encontrar nada.
Ejercicio 4. Notas	0,75	Los dos métodos del objeto, la validación (también la casilla vacía), los badges por color y la media con reduce.
Ejercicio 5. map, filter y reduce	0,75	Los tres métodos, el original sin cambios y los tres pasos encadenados con el resultado correcto.
Ejercicio 6. Ciudades	0,75	Las posiciones, slice frente a splice (lo que cambia en el array), restaurar, e includes e indexOf.
Calidad del trabajo entregado	1	Código limpio, comentado, sin funciones flecha y sin restos de la plantilla (huecos o TODO sin quitar); README cuidado, con las seis capturas propias comentadas y la reflexión; varios commits que muestran el avance.
Total	5	

Cada apartado se puntúa de 0 a su máximo, según lo que funcione, con cuartos de punto.

Defensa en clase (hasta +5)

La defensa vale **la mitad de la nota**: es donde demuestras que sabes hacer lo que has entregado. No se trata de repetirlo: te pediré uno o dos cambios pequeños, los harás en el momento explicando lo que haces y te preguntaré por tu código.

- **Cuándo:** en la sesión de clase que indique después del cierre del plazo. Unos cinco minutos por persona.

- **Requisito:** solo puedes defender la tarea si la has entregado en plazo. Si no la defiendes, tu nota es la del trabajo (máximo 5).
- **Cómo:** abres tu página desde la versión entregada, la enseñas funcionando con la consola abierta, haces en tu propio código el cambio que te pida y respondes a lo que te pregunte sobre él.
- **Sin IA:** durante la defensa desactiva Copilot o cualquier autocompletado con IA. Puedes consultar los apuntes y la documentación de MDN.

Ejemplos de lo que te puedo pedir

- Añadir al estudiante un método nuevo, por ejemplo `esMayorDeEdad()`, y usarlo en la ficha.
- Ordenar el catálogo de más caro a más barato, o por nombre.
- Cambiar el filtro del ejercicio 3, por ejemplo para quedarte con los productos de una categoría.
- Añadir al alumno un método que devuelva su nota más alta.
- Explicar qué pasaría si ordenaras `productos` sin copiarlo, o si `hacerSplice()` usara `slice`.
- Encontrar y corregir, con ayuda de la consola, un error que yo introduzca en tu código.

Cómo se puntúa

Puntos	Qué demuestras
5	Haces el cambio sin ayuda y con soltura, explicas con seguridad tu código y lo que aparece en la consola, y respondes bien a lo que te pregunto.
4	Haces el cambio sin ayuda, con alguna duda menor, y explicas bien lo esencial.
3	Haces el cambio con una pista pequeña y explicas correctamente la mayor parte de tu código.
2	Necesitas varias pistas para hacer el cambio o tu explicación tiene lagunas importantes.
1	No consigues hacer el cambio sin mucha ayuda y solo sabes explicar partes sueltas de tu código.
0	No consigues hacer el cambio ni explicar tu propio código, o no te presentas.

Se pueden dar valores intermedios de medio punto.

Nota final = trabajo (máximo 5) + defensa en clase (máximo 5).

Uso de la inteligencia artificial

Puedes apoyarte en herramientas de IA para orientarte, entender un error o revisar la redacción, pero el código y las explicaciones tienen que ser tuyos. Si las utilizas, indícalo al final del README en un apartado «Uso de IA»: qué herramienta, para qué y qué hiciste después con su respuesta. Declararlo no resta nota.

Advertencia: si se detecta que la tarea se ha generado con IA sin trabajo propio, la calificación será un 0 absoluto.

Son indicios: la falta de evidencias propias, el código con técnicas que no hemos visto y que no sabes explicar (por ejemplo, funciones flecha en esta tarea), los textos genéricos que no responden a lo que se pide, subir todo el trabajo de golpe en un único commit y no saber explicar lo entregado. Una captura tomada de internet, prestada o generada cuenta como el mismo caso.

La defensa en clase sirve precisamente para comprobarlo: si en ella no eres capaz de explicar nada de lo que has entregado, no solo no sumas esos 5 puntos, sino que se considera un indicio claro de que la tarea no es tuya.

Ejemplo de resultado esperado

